IMPLEMENTATION REPORT

Pediatric Clinical Database

JSON Dynamic Forms - Progress, Team Contribution and Hospital Integration

CURRENT OUTCOME

The dynamic-form subsystem has been replaced end-to-end with a versioned JSON architecture. Clinical records now use recoverable soft deletion, forms are selected per visit, and previous patient follow-up data is available during continuity-of-care work.

What is complete

- Core pediatric patient, visit, clinical-record, medication, laboratory and follow-up workflows.
- Four-table JSON form model: templates, template fields, responses and response history.
- Safe one-time conversion script for an existing database using the earlier EAV form tables.
- Draft-to-active template lifecycle, immutable published versions and retirement of the previous active version.
- Server-side answer validation, visit linkage, transactional upsert and an audit/history trail.
- Clinical Forms interface for form creation, draft cloning, field add/edit/delete, publication, response entry and history review.
- Reason-required soft deletion with actor, timestamp and reason for patients, visits, prescriptions, labs, follow-ups and other clinical records.
- Visit-specific form availability and saved response revisions, plus prior patient history in follow-up creation and review.
- Expanded demonstration data: 30 patients, 30 guardians, 7 doctors, 31 appointments and 17 completed visits.

Release position

ACADEMIC / EVALUATION	HOSPITAL PRODUCTION
Implemented and demonstrable with synthetic data. The JSON architecture and upgrade path are verified.	Not yet approved for real patient care. Identity, interoperability, infrastructure, governance and clinical validation remain.

01 JSON form architecture

The form definition is relational; flexible field rules and response payloads use validated JSON.

custom_forms	custom_form_fields	custom_form_responses	response_history
One row per template version Unique: form_key + version draft / active / retired	Field key, label, type Options and validation JSON Section and display order	One answer object per visit and form version Optional extra-fields array Current revision number	Snapshot for every save Who changed it and when No clinical overwrite is lost

Data flow

Example response document

```
{
"night_symptoms": "weekly",
"reliever_days": 4,
"activity_limited": true,
"assessment_note": "Review after two weeks"
}
```

Key design decisions

- A template version has its own form_id, so a historical response always points to the exact definition used.
- Published templates are read-only. Changes begin by cloning a new draft version.
- The API rejects unknown fields, missing required answers, invalid options and values outside numeric or length rules.
- Stable hospital concepts such as patients, visits, prescriptions and diagnoses remain normalized relational tables.

02 Progress register

Evidence reflects the code and verification completed for this handover.

AREA	STATUS	EVIDENCE
Relational hospital foundation	Complete	Patients, guardians, appointments, visits, clinical records, diagnoses, vitals, growth, allergies, immunizations, prescriptions, laboratory and follow-ups.
Patient workflows	Complete	Register and edit patients; maintain demographics, identifiers, guardians, consents and safety summary.
Clinical data workflows	Complete	Add and edit encounter data, prescriptions, allergies, immunizations, laboratory orders/results and follow-up activity.
JSON form schema	Complete	Four requested tables, JSON type checks, foreign keys, unique version constraints and indexes.
Existing-data upgrade	Complete	Migration 004 converts old template metadata and typed EAV values into the new JSON response model.
Dynamic form API	Complete	Create, clone, edit draft fields, publish, validate, save, update and read history.
Dynamic form interface	Complete	Navigation, visit selection, renderer, builder, publication and revision history.
Retention and deletion	Complete	17 clinical tables support soft deletion. Audit/history links restrict deletion; only three configuration cascades remain.
Continuity of care	Complete	Each visit has independent form responses; old patient follow-ups appear while creating and reviewing later work.
Automated verification	Complete	Fresh schema/seed test, deletion endpoint tests, template-delete guard, production UI build and browser workflow checks.

Work still required before a hospital deployment

WORKSTREAM	REQUIRED OUTCOME
Identity and authorization	Connect hospital SSO/identity provider; enforce role, department, patient-context and break-glass policies at every API endpoint.
Interoperability	Implement a tested FHIR/HL7 adapter, external identifier mapping, terminology mapping and reconciliation queue.
Clinical governance	Clinician sign-off for forms, ranges, alerts and workflows; define who may publish or retire a form.
Security and operations	Encryption and key management, secrets vault, immutable audit retention, backups, disaster recovery, monitoring and incident response.
Quality assurance	Unit/integration/E2E test suite, concurrency/load tests, accessibility review, validation on supported browsers and failure-mode testing.
Data governance	Consent, retention, correction, disclosure and de-identification policies aligned to the hospital jurisdiction.

03 Contribution plan for three people

Replace Member 1/2/3 with the actual names before academic submission.

MEMBER 1

Database and integration lead

- Own the MySQL schema, constraints, indexes, seed data and migrations.
- Validate the EAV-to-JSON conversion and write reconciliation queries.
- Design external identifier mapping and the hospital FHIR/HL7 adapter.
- Maintain data dictionary, ER model, backups and recovery runbook.

MEMBER 2

Backend, security and audit lead

- Own Express routes, services, transactions and answer validation.
- Implement template lifecycle, response revisions and audit events.
- Integrate SSO/RBAC, authorization checks, rate limits and secure configuration.
- Build API, security, concurrency and integration tests.

MEMBER 3

Frontend, testing and documentation lead

- Own React patient, clinical, follow-up and dynamic-form workflows.
- Implement form builder, renderer, visit selection and history interface.
- Run accessibility, browser and end-to-end testing with synthetic data.
- Maintain evaluation scripts, user guide, training material and handover report.

Shared responsibilities

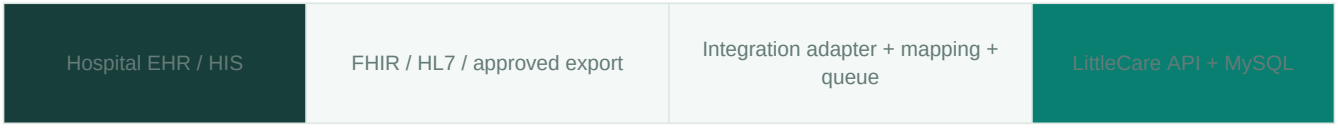
- Use pull-request review: at least one teammate reviews every schema, API or UI change.
- All three members participate in clinical workflow review, demonstration rehearsal and defect triage.
- Evidence should be tracked by commit, test result and document section - not only by a percentage split.

04 Integration with an existing hospital database

Use an interface layer and controlled identifiers; do not connect this prototype directly to production tables.

1	**Discover and map** Inventory the hospital EHR/HIS interfaces, supported FHIR version, HL7 feeds, master patient index, visit identifiers, code systems and source-of-truth ownership.
2	**Create an integration boundary** Place an adapter or interface engine between the hospital system and LittleCare. Prefer supported APIs/events; avoid direct writes into vendor-owned tables.
3	**Map identity** Maintain external-system + external-id mappings for Patient, Encounter/Visit, Practitioner and Organization. Use deterministic matching only where approved; queue ambiguity for human reconciliation.
4	**Map clinical resources** Typical mappings: Patient -> patients; Encounter -> visits; Observation -> vital signs/labs/form answers; Condition -> diagnoses; MedicationRequest -> prescriptions; Appointment -> appointments.
5	**Secure the connection** Use SMART/OAuth where supported, least-privilege service accounts, TLS, network allowlists, secret rotation, user context and end-to-end correlation IDs.
6	**Pilot safely** Test with synthetic data, then a read-only shadow feed, reconciled pilot cohort and monitored phased rollout. Define rollback, downtime and manual-continuity procedures before go-live.

Recommended deployment pattern



Important integration rule

If the hospital and LittleCare share one governed MySQL environment, the visit_id foreign key can link forms directly to visits. If the hospital database is external, keep local surrogate keys and store approved external identifiers in a mapping table. Never assume that a hospital MRN or encounter number is globally unique without its assigning system.

05 Verification, handover and next milestone

The checks below were executed against an isolated MySQL instance containing synthetic data.

AREA	STATUS	EVIDENCE
Fresh database and seed	PASS	Schema, triggers and workflows applied. Verified: 30 patients, 30 guardians, 7 doctors, 31 appointments and 17 visits.
Retention rules	PASS	17 soft-delete tables; only three approved configuration cascades; active/retired form deletion blocked by trigger.
Legacy upgrade	PASS	Old form/field/value rows converted to new template fields and a typed JSON answer object.
Code and UI build	PASS	Node syntax checks passed and the Vite production build completed successfully.
Template lifecycle	PASS	Clone active version, edit draft, publish new version and retire the prior version; active edits return HTTP 409.
Response lifecycle	PASS	Visit-linked create/update/history and delete/restore revision behavior verified.
Clinical soft deletion	PASS	Visits, allergies, immunizations, prescriptions and labs retained deletion metadata and wrote audit events.

Primary handover files

`database/migrations/001_schema.sql`	Fresh-install JSON form schema
`database/migrations/004_json_dynamic_forms.sql`	One-time upgrade from the earlier EAV model
`database/migrations/005_soft_delete_and_retention.sql`	Existing-database retention upgrade
`server/src/routes/forms.js`	REST endpoints and transactions
`server/src/routes/clinical.js`	Clinical create, edit and soft-delete operations
`server/src/services/forms.js`	JSON parsing, grouping and answer validation
`client/src/pages/Forms.jsx`	Form builder, renderer, visit entry and history
`client/src/pages/FollowUps.jsx`	Current and previous follow-up workflow
`database/queries/evaluation_queries.sql`	JSON reporting demonstrations

Recommended next milestone

Build the hospital integration sandbox: add external identifier tables, select the target hospital interface and FHIR version, implement read-only Patient/Encounter import, enforce SSO/RBAC, and produce a reconciliation report using synthetic records. Do not begin production data transfer until hospital security, privacy and clinical governance approve the design.

Standards references

HL7 FHIR R5: https://hl7.org/fhir/
SMART App Launch 2.2: https://hl7.org/fhir/smart-app-launch/

PRODUCTION SAFETY NOTE

This is a tested academic prototype using synthetic data, not a certified medical device or a production EHR. Hospital deployment requires formal technical, clinical, security, privacy and regulatory approval.